

The Isomorphic Scheduler: A Machine-Checked Conservation Law for Concurrency, Distribution, and Correctness

Fabian Franz
Isomorphic AI

Claude (Anthropic)
Research Collaborator

Version 1.0.0 — July 19, 2026

Abstract

Concurrency control, distributed consistency, scheduling, and program correctness are practised as four fields with four toolkits and four famous impossibilities. We show they share one structure: **a single conserved quantity Q moving through a graph of dependencies, mishandled in exactly one way—a claim held where it cannot convert—and cured in exactly one way—routed to where it converts, or released.** The quantity has three readings related by calculus: its flow (a scheduling rate), its stock (a detection budget, the integral of flow), and its stock banked across a discontinuity (a resolution credit). We state four conservation laws and formalize the theory in Lean 4: 504 theorems, no `sorry`, no axioms beyond the kernel, including conservation under every transition class, absorption of deadlock, exactness of the stock–flow integral, soundness and budget-bounded completeness of deadlock detection over an explicit wait-for-graph model, victim-free resolution losing no work, the hoarding theorem over a finite certified policy grammar, and the equivalence of clock-free invariance with conservation. From conservation, three system invariants follow as corollaries rather than design goals—evidence tracking, shared accounting, and no private gain at the whole’s expense—and the classical impossibilities (victim-based deadlock recovery, priority-inversion protocols, CAP as a simultaneous demand, the safety/liveness trade-off) dissolve as boundary or simultaneity artifacts. Conservation itself is elementary physics; the contribution is its transfer: identifying the scheduler’s budget, credit, and rate as one conserved quantity and deriving the field’s classical mechanisms as its corollaries. Seven clock-free verifier engines and the Lean development constitute the evidence.

1 Problem Revisited

A practitioner who learns deadlock handling, then distributed consistency, then real-time scheduling, then correctness proofs, acquires four toolkits that appear unrelated. Deadlock handling reaches for timeouts and victim selection. Distributed systems reach for quorums and accept eventual consistency as a tax. Schedulers reach for priority inheritance protocols that boost and restore. Correctness arguments reach for the safety/liveness dichotomy. Each field carries an impossibility it treats as a fact of nature, and each impossibility motivates a defensive mechanism whose tuning is a chronic source of bugs: a timeout short enough to catch deadlocks aborts slow-but-live work; long enough to protect it, it misses deadlocks.

The question this paper answers: are the four toolkits genuinely distinct, or four expressions of one structure? If one structure, three things follow. The same primitive should solve all four. The impossibilities should be artifacts of how each problem is posed. And the chronic tuning bugs should be symptoms of measuring the wrong quantity—time, the shadow, instead of the conserved quantity itself. We establish all three constructively, and—new relative to the working series this paper condenses—mechanically: the laws are now theorems in Lean 4, and the formalization sharpened one of them.

2 Background

No prior knowledge of any of the four fields is assumed; this section is the whole of the on-ramp. Each idea takes one paragraph, and each will be needed exactly once.

Conservation. Some quantities move and transform but never appear or vanish: pour water between glasses and the total is the total. Physics has run on this for centuries, and modelling disciplines made it exact bookkeeping—a *stock* is simply the running sum of what has flowed in minus what has flowed out (Forrester, 1961 [industrial-dynamics]), and a conserved quantity can be routed along the edges of a graph, balancing at every node (Ford & Fulkerson, 1962 [flows-in-networks]). Nothing in the law is new here; hold on to the water-between-glasses picture, because the rest of this paper is that picture pointed at software.

Deadlock. Two programs each hold something the other needs, and each waits for the other: neither will ever move. Drawing who-waits-for-whom as a graph and looking for a cycle is the classical recognition (Coffman, Elphick & Shoshani, 1971 [coffman-conditions]; Holt, 1972 [deadlock-graphs]). The delicate part has never been the cycle—it is deciding that a slow program is actually *stuck*, which systems traditionally guess from elapsed time. And once stuck, the traditional cure is blunt: abort one participant and throw its half-finished work away (Bernstein, Hadzilacos & Goodman, 1987 [concurrency-control]; Rosenkrantz, Stearns & Lewis, 1978 [wound-wait]). Both the guess and the discard are assumptions this paper removes.

Priority inversion. An urgent task can be made to wait behind an unimportant one, if the unimportant one holds a lock the urgent one needs while a third task hogs the processor. This once rebooted a spacecraft on Mars (Reeves, 1997 [pathfinder]). The standard fix temporarily promotes the lock holder to the urgency of its most impatient waiter, then demotes it afterwards (Sha, Rajkumar & Lehoczky, 1990 [priority-inheritance]); a gentler family of schedulers simply shares the processor in proportion to weights (Demers, Keshav & Shenker, 1989 [fair-queuing]; Waldspurger & Weihl, 1994 [lottery], 1995 [stride]). Keep the word *proportion*; it returns.

Distribution. When one system runs on many machines, the network can split them into islands that cannot talk. A famous result says a split system cannot be simultaneously consistent (all islands agree) and available (all islands keep working) (Brewer, 2000 [cap]; Gilbert & Lynch, 2002 [cap-proof])—and the word *simultaneously* will matter later. The practical peace treaty is eventual consistency: let the islands drift, guarantee they reconcile (Vogels, 2009 [eventually-consistent]); data structures whose merge is order-independent make the reconciliation automatic (Shapiro et al., 2011 [crdts]; machine-checked by Gomes et al., 2017 [verifying-crdts]).

Correctness and time. Program properties come in two kinds: nothing bad ever happens (safety) and something good eventually happens (liveness) (Lamport, 1977 [proving-correctness]; Alpern & Schneider, 1985 [defining-liveness]). Across machines even “before” and “after” are relative (Lamport, 1978 [time-clocks]), and a network call does not have a duration—it has a *distribution* of durations with a long tail that practical systems hedge against (Dean & Barroso, 2013 [tail-at-scale]); assuming otherwise is the first of the classic fallacies of distributed computing (Deutsch, 1994 [fallacies]).

That is the whole background. What none of these literatures states is that the deadlock budget, the discarded work, the promoted priority, the drifting islands, and the collapsed distribution are appearances of *one* conserved quantity, mishandled the same way each time. That statement is this paper.

3 One Quantity, One Fault, One Cure

A system is a graph of dependencies. Through it moves one conserved quantity Q : moved and converted, never created or destroyed, with conversion to progress as the sole true sink. Conservation is meant in the strong, checkable sense: the total—unallocated reserve, plus held stock, plus converted work at its conversion cost—is invariant under every operation.

There is exactly **one fault**: Q held where it cannot convert—a claim parked at a node unable to turn it into progress. A deadlock is Q held in a cycle where no member can convert. A priority inversion is urgency held by a blocked task while the holder it depends on goes underfunded. A premature commitment is the decision-quantity collapsed to an actuality before the evidence needed to convert it has arrived. The fault is always a *stranded claim*.

There is exactly **one cure**: route Q to where it can convert, or release it. Detection recognises the stranded claim; resolution releases it while conserving the releaser’s work; scheduling routes it to the node that can convert it; distribution preserves it across a partition until reconciliation; correctness reads whether conversion is happening; flexibility withholds a collapse until an outcome can convert the decision.

The dissolution move. Each field’s impossibility, examined under this structure, is an artifact of a boundary drawn too small or a simultaneity demanded too strictly. The victim in deadlock recovery assumes the releaser’s work must be discarded; the boost-and-restore protocol treats priority as a stored claim rather than a flow; CAP demands its three properties as a simultaneous snapshot; the safety/liveness trade-off measures duration, which cannot distinguish slow from dead. Widen the boundary to the whole, replace the snapshot with a schedule over time, measure the conserved quantity rather than its shadow—and the impossibility is local, not fundamental.

The structure has a load-bearing consequence:

Theorem 1 (Hoarding is self-defeating). *In a connected dependency graph, a node that hoards the conserved quantity—holding a claim it cannot itself convert rather than routing or releasing it—starves the nodes it depends on, which hold the resources it needs, and therefore obstructs itself. The selfish optimum and the generous optimum coincide: routing Q to where it converts is simultaneously the best move for the whole and for the part.*

The seven readings of Section 4.4 are its instances. The theorem itself is machine-checked: for a stranded claim, routing strictly dominates hoarding in the holder’s *own* conversion, and over a finite certified grammar of claim policies the selfish-optimum and generous-optimum predicates are provably equal

route_stranded_claim_strictly_dominates_hoard,
selfish_optima_eq_generous_optima_in_finite_certified_policies.

The proven scope is that named policy grammar—strict dominance and obstruction, not a claim over all conceivable strategies.

4 The Theory

4.1 The model

Every claim in this paper is decided by the definitions of this half-page; the Lean development contains nothing that is not derived from them. We reproduce them in mathematical dress. A system over n processes is $s = (\text{procs}, \text{runnable}, R, \kappa, Q_{\text{tot}})$ where each process p_i carries a stock $S_i \in \mathbb{Q}_{\geq 0}$, a banked credit $C_i \geq 0$, a converted-work count $W_i \in \mathbb{N}$, and a net-flow integral I_i ; $R \geq 0$ is the unallocated reservoir and $\kappa > 0$ the conversion cost. The accounted total is

$$\text{acc}(s) = R + \sum_i (S_i + C_i + \kappa W_i).$$

Three transition classes act on s , each specified as a *plan* carrying its own non-negativity obligations (so well-formedness is part of the type, not an afterthought):

- **Flow steps** inject shares from R into runnable processes and convert stock to work at cost κ per unit (blocked processes receive and convert nothing);
- **Drain steps** return stock from blocked processes to R (runnable processes are untouched by hypothesis);
- **Yield steps** move a process’s stock into its banked credit—the integral carried across a restart discontinuity.

Well-formedness $WF(s)$ packages positivity of κ , non-negativity of all stocks, credits, rates, and the reservoir, and the ledger identity $acc(s) = Q_{tot}$; it is proven preserved by all three transition classes and by arbitrary reachable compositions of them.

4.2 The four laws and their proof status

Law 1 (L1, Conservation; *proven*). *For every transition class, $acc(step(s)) = acc(s)$. Flow moves Q from the reservoir to stocks and from stocks into converted work; draining moves it back to the reservoir; yielding moves it from stock to credit. No operation creates or destroys Q .*

Law 2 (L2, Monotonicity; *proven*). *Along any reachable sequence of non-injecting transitions (drain and yield steps), the convertible stock $\sum_{i \text{ runnable}} S_i$ is monotone non-increasing; moreover each blocked process’s stock is individually non-increasing under arbitrary mixed sequences, while flow steps leave blocked stocks unchanged.*

Law 3 (L3, Absorption; *proven*). *A deadlocked state (no process runnable, not all done) is a fixed point of the dynamics: any further step preserves the state and its converted totals. In the explicit wait-for-graph model, a closed wait-set—each member blocked on a lock held inside the set—remains closed under iteration, converts nothing forever, and its members never complete.*

Law 4 (L4, Stock is the integral of flow; *proven*). *For every process and every reachable state, $S_i + C_i = I_i$: held stock plus banked credit equals the running integral of net flow, exactly. Rate is the derivative of this stock; credit is the integral banked across a yield.*

L4 holds over the same transition system as L1–L3, via an executable-trace bridge (`core_reachable_has_trace`) so the integral and the dynamics it integrates are one object. L4 is the unification made precise: the detection budget is $\int \text{rate } dt$, the resolution credit is that integral carried over a discontinuity, and the scheduling rate is its derivative. Three mechanisms, one quantity, related by calculus.

Beyond L1–L4 the development proves three families of results worth naming. *Detection, both directions*: if the budget-floor-plus-closed-cycle detector fires, a genuine deadlock exists (`detection_sound`); every deadlock is detected within a latency bounded by the budget (`detection_complete_budget`); and a live, converting process is never detected (`live_process_not_detected`). *Share-sum*: proportional shares computed from weights over the runnable set sum exactly to the scheduling quantum, the lemma that decouples conservation from the details of any particular flow equation. *Invariance is conservation*: a quantity is a clock-free invariant of the mixed dynamics if and only if it is a conserved charge (`clockFreeInvariant_iff_conservedBy`, instantiated by the accounted total)—the paper’s “no wall clock anywhere” as an equivalence rather than a design preference. The restriction of L2 to the non-injecting class is essential— injection into a stalled runnable process can raise convertible stock without progress; the evolution of the statement is recorded in a companion note [iso-theory-history].

4.3 The three invariants as corollaries

From L1, three system-design invariants follow as corollaries rather than as independent goals.

Corollary 1 (Epistemic invariant). *Because Q cannot be conjured, the books must balance; every quantity is measured and accounted, never asserted. Concretely: conservation is a checked invariant at every step, and a distribution over outcomes is not collapsed to a point before the evidence that converts it has arrived (Section 4.4, reading 7).*

Corollary 2 (Alignment invariant). *Because Q moved between parts is conserved—one part’s decrease is exactly another’s increase—a part’s success runs through the whole’s. Merging unions contributions rather than overwriting them; funding a node on one’s own critical path is routing, not sacrifice.*

Corollary 3 (Agency invariant). *Because conservation forbids private creation of Q , no action yields a private gain at the whole’s net expense: every gain is a transfer, and the only net sink is conversion to progress, which is the shared goal.*

A small formal companion sharpens the epistemic invariant in an unexpected direction: in the **Polarity** module, trust extended to *recorded presence* (a cached proof) is monotone under information loss, while trust extended to *absence* (“no record, so it never happened”) admits a machine-checked counterexample. Trust what the ledger contains; never trust what it merely omits.

4.4 Seven readings

The four laws, read in seven registers, recover the four fields’ mechanisms and dissolve their impossibilities. Each reading is verified by a dedicated clock-free engine (Section 6).

(1) Detection: read the stock’s floor, not a clock. Give each process a budget (its stock); conversion resets it, non-conversion drains it. Budget floor plus a closed wait-cycle is the detector. A slow-but-live process keeps converting and never floors; only genuine non-conversion drains to the floor. Timer-free detection with zero false positives and latency bounded by budget size—soundness proven (`detection_sound`).

(2) Resolution: bank the integral; no victim. The yielding process’s converted progress is banked as credit (L4’s integral carried across the restart), so it returns with base plus credit and converges instead of relitigating the same contention. Verified head-to-head: credit-conserving yield completes in two yields losing zero work where discard-the-work livelocks through hundreds of aborts.

(3) Scheduling: route the derivative; inheritance for free. Priority is a rate, not a stored claim. A blocked process’s rate flows down its wait-edge: $\text{eff}(p) = \text{base}(p) + \sum \text{eff}(w)$ over waiters w blocked on p , transitively. The holder of a contended lock is funded in exact proportion to the urgency waiting on it; on release the wait-edge vanishes and rates revert, memorylessly. Inversion cannot form. The divergence from classical inheritance is the *sum* rather than the maximum—which is what conservation requires, since the waiters’ rates must go somewhere.

(4) The law itself. Section 4.2 is the fourth reading: the quantity stated and proved rather than used. It is the hinge—remove it and readings 1–3 are three mechanisms that happen to rhyme; with it, budget, credit, and rate are forced to be one object by L4, and the remaining readings inherit the proofs.

(5) Distribution: carry Q across partition; CAP is a scheduling problem. Represent Q as a PN-counter (per-node grow-only contributions; merge = per-node supremum, a semilattice join). Under partition, Q is hidden, not destroyed; what a node spends off the books surfaces exactly on heal, and convergence is order-independent. The deeper move: CAP’s three properties are the three corollaries of Section 4.3. Consistency is the epistemic invariant; availability is the agency invariant; partition tolerance is the alignment invariant. The impossibility arises only

when the three are demanded as a simultaneous snapshot—the “at once” of Section 2. Delivered as a schedule, they reinforce: partition is the normal state, consistency is deferred to a sync triggered by inventory mismatch rather than a clock, and availability is pointwise, one atomic reversible action at a time.

(6) Safety and liveness: measure conversion, then ask. Duration cannot distinguish slow-but-live from dead; conversion can. Safety follows from monotone blocked stocks (a live process converts, refills, never floors); liveness from absorption (a dead set drains to the floor in bounded steps). Same fact, both properties, nothing to trade. For the one case external measurement cannot see—full CPU, zero output—stop inferring and ask: a progress signal answered by a monotone counter refills the budget; a flat counter drains it. A process can lie, but a lie corrupts the liar’s own ledger and is caught by comparing reported to delivered progress.

(7) Flexibility: Q moving from potential to actual. A network link has a latency distribution, not a latency. Collapsing the distribution to a point estimate—an assumed completion order—is the stranded-claim fault in probabilistic dress: it manufactures phantom races that were always latent in the tail. Carry the distribution and branch on the order that actually resolves, and the race class cannot form. Flexibility appears non-conserved (a collapse only spends it); accounting both sides shows the collapse converts potential into actuality one-for-one—L4 across the potential/actual boundary. What is the agent’s to choose is not whether the decision moves but *when*: on evidence, never on a guess or a timer.

5 Related Work

Deadlock. The deadlock literature has developed successive methods for deciding that a system is stuck—cycle tests over the wait-for graph [coffman-conditions, deadlock-graphs], triggered in practice by elapsed time. Our approach shows that the effective measure is work done, not time elapsed: with conversion as the signal, a slow process and a dead one are distinguished at the same budget size, soundness is a theorem (**detection_sound**), and the safety/liveness trade-off does not arise. For recovery, the literature aborts a victim and discards its partial work [concurrency-control, wound-wait]; banking that work as conserved credit resolves the same deadlocks with no victim, and in direct comparison the discard variant livelocks where the banked variant completes in two yields.

Scheduling. The inheritance literature bounds priority inversion by raising a lock holder to the maximum urgency among its waiters [priority-inheritance, fair-queueing, lottery, stride]. Conservation requires the sum: the waiters’ rates must go somewhere, and they go to the holder. With the sum, inversion-freedom is structural and memoryless—there is no boost to apply and no restore to mistime.

Distribution. The CAP theorem shows that consistency, availability, and partition tolerance cannot hold at one instant [cap, cap-proof]; the practical response relaxes consistency [eventually-consistent], with CRDTs supplying coordination-free convergence [crdts, verifying-crdts]. Read through conservation, the three properties are the three corollaries of Section 4.3, and they hold as a schedule: partition as the normal state, consistency at each sync, availability at each atomic action. The same system is consistent, available, and partition-tolerant—at different moments.

Correctness and tails. The tail-at-scale literature hedges against latency distributions [tail-at-scale], and the fallacies catalogue records the recurring contrary assumptions [fallacies]. Both practices follow from one mechanism: committing a distribution to a point before evidence arrives is a stranded claim, and it manufactures the races that appear under load. A system that branches only on resolved order has no assumption for load to violate [time-clocks].

Machine-checked systems theory. CRDT convergence has been verified in a proof assistant [verifying-crdts]. The present development verifies a conservation law from which detection soundness and completeness, victim-free resolution, absorption, the stock–flow identity, and the hoarding theorem follow together.

The unifying claim. Each literature above solves its problem with a mechanism specific to its field. Under the conservation reading, the mechanisms are corollaries of one law, and the impossibilities—the victim, the boost-and-restore protocol, CAP at an instant, the detection trade-off—do not survive the widened boundary. The claim is falsifiable twice over: exhibit a pathology in these domains that is not a stranded claim, or an impossibility that survives widening the boundary to the whole. We pre-register it and invite the attempt.

6 Evaluation

Two independent bodies of evidence: seven clock-free numerical engines (exact arithmetic or exhaustive enumeration; audited to contain no wall-clock primitive), and the Lean 4 development.

Reading	Engine	Verifies
1 Detection	<code>iso_deadlock.py</code>	floors+cycles catch deadlocks/knots; 0 false pos.; bounded lat
2 Resolution	<code>iso_resolve.py</code>	credit-yield loses 0 work in 2 yields; discard-work livelocks
3 Scheduling	<code>iso_flow.py</code>	blocked rate reaches holder (0.77 share); memoryless return
4 Conservation	<code>iso_conserve.py</code>	L1 invariant incl. routing; L2 drain 18→0 monotone; L3; L4 e
5 Distribution	<code>iso_distributed.py, iso_cap.py</code>	partitioned views converge; debt surfaces on heal; order-independent; failure paths preserve content
6 Safety/liveness	<code>iso_safety.py, iso_progress.py</code>	exhaustive: 0 false pos., 0 false neg.; progress signal keeps useful, reclaims spinner, catches liar
7 Flexibility	<code>iso_flex.py</code>	premature collapse races on tail; carried distribution: 0 races; potential+actual conserved across collapse

Table 1: The seven engines. All **PASS**; each reproduces deterministically; **grep** audit confirms no time/sleep/clock primitive in any engine.

Module	Content	Key results
Basic, L1–L4	the four laws	conservation, monotonicity, absorption, stock–flow exactness under all transitions
Reachable, CoreTrace	closure and traces	laws carried over arbitrary reachable mixed sequences and executable core traces
WaitGraph, BudgetWait, DetectorProgress	detection	detection_sound ; detection_complete_bounded ; latency $\leq$ budget; live processes never detected; progress-signal keep/reclaim
ResolutionYield	resolution	yield conserves, loses no work, breaks the closed component
RateRouting, ShareSum	scheduling	routed-rate conservation (KCL form); shares sum exactly to the quantum
PNCounter	distribution	merge convergence and conservation across partition
TheoremOne	the hoarding theorem	strict dominance of routing; selfish and generous optimum predicates equal over the certified grammar
Noether	invariance	clock-free invariance $\Leftrightarrow$ conserved charge
Canonical, PaperClaims, Polarity	bridges	Python engine step verified against the model; the papers’ claims restated as lemmas; presence-trust monotone, absence-trust refuted

Table 2: The Lean 4 development: 8,766 lines, **504 theorems/lemmas, zero sorry, no axioms beyond the kernel** (branch `iso-conserve-lean` [iso-conserve-lean]).

Three readings of the evidence. First, the division of labour: the engines demonstrate the *phenomena* (livelock under discard, the 0.77 routed share, phantom races on the tail); the Lean proves the *laws* the phenomena instantiate; and the two are pinned together by

`canonical_python_step_verified`, which checks the engine’s step against the formal model. Second, the L2 restriction (Section 4.2) is evaluation in the strongest sense—the formalization falsified a clause of the prose theory and returned the repaired clause with a proof; the evolution is conserved in the companion note [iso-theory-history]. Third, scope honesty: the laws are proven for the closed fluid model and the wait-graph model as defined; open systems with external sources and sinks, and the quantitative rate of eventual consistency, are stated as open problems, not claimed.

In the field. The detection reading now runs in production shape: the same floor-plus-cycle predicate operates inside MySQL 8.0.46 (a 10 ms epoch plane, roughly two sealed epochs to declare, overhead below a two-sigma noise floor under real Drupal HTTP traffic), has been transferred to PostgreSQL as a sidecar plus an extension that instruments its own conversion signal, and to SQLite at transaction grain; two independent implementations have co-witnessed one live cycle. No conversion signal survived an engine boundary; the transition consuming the signals is character-for-character identical in every implementation. The ports and the induced criterion for what counts as a conversion signal are reported in a companion paper [conversion-budgets]. The falsifiability invitation of Section 5 is therefore concrete: the thing the theorems describe can be run.

7 Further Work

Open systems. The law is proven for a closed world. Real systems have births and deaths: spawned processes inject demand, killed processes surrender stock. Does a continuity equation with accounted source and sink terms preserve the debt-surfacing behaviour of Section 4.4, or does openness admit genuine leaks? Where must the boundary be drawn for the books still to balance?

The kernel. It remains to be seen whether the flow equation—summed along the wait-graph, not maximised—can replace the weight computation of a production fair scheduler without disturbing the conservation-like invariants (lag, eligibility) the scheduler already maintains, and what problems appear when it meets real workloads under induced inversion and induced tail latency; a vehicle for the experiment now exists (a BPF port on User Mode Linux). We are curious whether what breaks first is the implementation or the theory.

Structure of the graph. If rate on wait-edges is a network flow, is the deadlocked set a min-cut? Does max-flow name the maximal schedulable throughput around a contended resource? The correspondence is suggestive and, so far, unexamined. In the same spirit: under a gossip model with fixed fanout, can “eventually consistent” be bounded the way detection latency is—is *eventually* a number?

A second law? If flexibility is defined as the entropy of the live distribution, is the directional view of Section 4.4 a genuine second law for systems—an arrow—or the first law read as a schedule? We have not been able to settle this, and we would like to know.

Byzantine contributions. The PN-counter assumes each node honestly records its own contribution. Can a merge detect a contribution that violates conservation? What is the conserved-quantity analogue of Byzantine fault tolerance?

Other substrates. The law does not mention processors. Where else does a pipeline that selects among alternatives carry a distributional quantity that can be destroyed before evidence arrives? One such transfer, to inference in probabilistic models, is being developed in a companion line of work; we suspect it is not the only one.

8 Conclusion

Four fields, four toolkits, four impossibilities—and one structure beneath them: a conserved quantity moving through a graph of dependencies, mishandled only ever as a claim held where it cannot convert, cured only ever by routing or release. The quantity’s three calculus readings recover detection (the integral’s floor), resolution (the integral banked across a yield), and scheduling (the derivative routed along the wait-edge); carried across partition it reframes CAP as a schedule; measured as conversion it dissolves the safety/liveness trade-off; watched moving from potential to actual it dissolves phantom races and its own apparent non-conservation. The laws are no longer narrative: 504 machine-checked theorems, including detection soundness and completeness, with the formalization sharp enough to have corrected the theory it formalized. Conservation is old physics; pointing it at schedulers is the contribution—and once pointed, the field’s defensive mechanisms stop being mechanisms to implement and become corollaries to read off. A system is well-run exactly when it holds no claim it cannot convert, and routes what it has to where the whole can use it.

References

- [1] J. W. Forrester. *Industrial Dynamics*. MIT Press, 1961. [industrial-dynamics]
- [2] L. R. Ford, D. R. Fulkerson. *Flows in Networks*. Princeton University Press, 1962. [flows-in-networks]
- [3] E. G. Coffman, M. Elphick, A. Shoshani. System Deadlocks. *ACM Computing Surveys*, 1971. [coffman-conditions]
- [4] R. C. Holt. Some Deadlock Properties of Computer Systems. *ACM Computing Surveys*, 1972. [deadlock-graphs]
- [5] P. A. Bernstein, V. Hadzilacos, N. Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. [concurrency-control]
- [6] D. J. Rosenkrantz, R. E. Stearns, P. M. Lewis. System Level Concurrency Control for Distributed Database Systems. *ACM TODS*, 1978. [wound-wait]
- [7] L. Sha, R. Rajkumar, J. P. Lehoczky. Priority Inheritance Protocols: An Approach to Real-Time Synchronization. *IEEE Transactions on Computers*, 1990. [priority-inheritance]
- [8] G. E. Reeves. What Really Happened on Mars? Mars Pathfinder priority inversion account, 1997. [pathfinder]
- [9] A. Demers, S. Keshav, S. Shenker. Analysis and Simulation of a Fair Queueing Algorithm. *SIGCOMM*, 1989. [fair-queueing]
- [10] C. A. Waldspurger, W. E. Weihl. Lottery Scheduling: Flexible Proportional-Share Resource Management. *OSDI*, 1994. [lottery]
- [11] C. A. Waldspurger, W. E. Weihl. Stride Scheduling: Deterministic Proportional-Share Resource Management. MIT Technical Report, 1995. [stride]
- [12] E. Brewer. Towards Robust Distributed Systems. PODC keynote (CAP conjecture), 2000. [cap]
- [13] S. Gilbert, N. Lynch. Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 2002. [cap-proof]

- [14] W. Vogels. Eventually Consistent. *Communications of the ACM*, 2009. [eventually-consistent]
- [15] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-free Replicated Data Types. *SSS*, 2011. [crdts]
- [16] V. B. F. Gomes, M. Kleppmann, D. P. Mulligan, A. R. Beresford. Verifying Strong Eventual Consistency in Distributed Systems. *OOPSLA*, 2017. [verifying-crdts]
- [17] L. Lamport. Proving the Correctness of Multiprocess Programs. *IEEE Transactions on Software Engineering*, 1977. [proving-correctness]
- [18] B. Alpern, F. B. Schneider. Defining Liveness. *Information Processing Letters*, 1985. [defining-liveness]
- [19] L. Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 1978. [time-clocks]
- [20] J. Dean, L. A. Barroso. The Tail at Scale. *Communications of the ACM*, 2013. [tail-at-scale]
- [21] P. Deutsch. The Fallacies of Distributed Computing, 1994. [fallacies]
- [22] F. Franz, Claude. *The Isomorphic Scheduler Series, Papers I–IX, with verifier engines*. Isomorphic AI, 2026. <https://github.com/isomorphic-ai/scheduler> [isomorphic-scheduler]
- [23] F. Franz, Claude. *IsoConserve: Lean 4 Formalization of the Conservation Laws*. Isomorphic AI, 2026. <https://github.com/isomorphic-ai/scheduler/tree/iso-conserve-lean> [iso-conserve-lean]
- [24] F. Franz, Claude. *Conversion Budgets in Production Databases: InnoDB, PostgreSQL, SQLite* (companion paper, in preparation). Isomorphic AI, 2026. [conversion-budgets]
- [25] F. Franz, Claude. *IsoConserve: Statement History of the Conservation Laws* (companion note). Isomorphic AI, 2026. [iso-theory-history]

A Reproducibility

Engines. Each engine runs as `python3 <engine>.py` and prints its checks and a verdict. Determinism: exact rational arithmetic or exhaustive enumeration; no randomness in any verifier. Clock audit: `grep -niE "time|sleep|clock|timeout|perf_counter|monotonic" <engine>.py` returns only prose in comments, for every engine.

Lean. Branch `iso-conserve-lean` of the series repository; built with Lean 4 via the provided `lakefile`. The development imports only `Std`. Audit: `grep -rn sorry` over the sources returns nothing; no `axiom` declarations are present. 504 `theorem/lemma` declarations across the 21 modules grouped in Table 2.

B The Law on One Screen

ONE conserved quantity Q in a graph of dependencies.

ONE fault: a claim held where it cannot convert.

ONE cure: route Q to where it converts, or release it.

<code>flow</code>	<code>= dQ/dt</code>	<code>-> routed along the wait-edge</code>	<code>= SCHEDULING</code>
<code>stock</code>	<code>= INT flow</code>	<code>-> its floor</code>	<code>= DETECTION</code>

credit = INT flow, banked across a yield	= RESOLUTION
the law itself (L1-L4, machine-checked)	= CONSERVATION
carried across partition (hidden, not lost)	= DISTRIBUTION
read as conversion present/absent	= SAFETY/LIVENESS
moving from potential to actual	= FLEXIBILITY

THREE invariants, corollaries of L1:

epistemic	-- account everything; collapse nothing before evidence
alignment	-- shared, not replacing; success runs through the whole
agency	-- no private gain at the whole's expense

THEOREM: in a connected graph, hoarding is self-defeating --
the selfish optimum and the generous optimum are the same point.

No wall clock anywhere.